Western Engineering

Electrical and Computer Engineering

AISE 4030 Reinforcement Learning

RL Project – Phase 02

Released on: Tuesday, February 25th, 2026
Due on: Sunday, March 8th, 2026

Note: Due date includes 48 hours grace period.

Table of Contents

1. Assignment Overview 1
2. Objectives 1
3. Assignment Details 1
   3.1. Task 1: Environment Setup & API Confirmation 1
   3.2. Task 2: Baseline Learning Agent Skeleton 2
      3.2.1. Documentation Requirements 2
      3.2.2. Code Structure & Naming Conventions 3
      3.2.3. Required Skeleton Components 5
4. AI Usage Policy 5
5. Submission Details 6
6. Grading Criteria 6
7. Team Collaboration 7

1. Assignment Overview

This second phase of the project is a technical milestone designed to transition your team from planning to implementation. The goal is to build the skeleton for your project: a functional environment, a working main loop, and a clear code structure for your learning agents.

You will not be graded on the performance of a learning agent in this phase. You will be graded on your ability to successfully set up your chosen environment, interact with its API, and create a well-structured code skeleton for your baseline learning agent. This step is crucial for de-risking your project and ensuring you have a solid foundation for Phase 3.

Two Algorithm Comparison: Your project must include at least **two RL implementations**. You will use a **base algorithm** and compare its performance against a more **advanced algorithm**. Examples:

- DQN (base) vs. Double DQN or Rainbow DQN (advanced)
- REINFORCE (base) vs. PPO (advanced)
- PPO (base) vs. SAC (advanced) for continuous control
- Any other pairing that represents a clear increase in algorithmic sophistication

In this phase, you are building the skeleton for your *base algorithm only*; the advanced algorithm will be implemented in Phase 3.

2. Objectives

Upon successful completion of this assignment, you will be able to:

- Install, configure, and run your chosen simulation environment.
- Interact with the environment's core API using Python.
- **Confirm the exact State and Action spaces** of your environment, verifying the claims from your Phase 1 proposal.
- Implement a full episode loop that can run from a start state to a terminal state.
- Create a clean, organized code skeleton (classes, modules, and methods) for your baseline learning agent.
- Structure your codebase so that it can be extended to support the advanced algorithm in Phase 3.

3. Assignment Details

This phase consists of two tasks: confirming your environment setup (Task 1) and building your agent skeleton (Task 2).

3.1. Task 1: Environment Setup & API Confirmation

Before you can build an agent, you must prove that your chosen environment is installed, running, and that you can interact with its API programmatically. Rather than writing a throwaway test script, you will build this verification directly into your project code — specifically your environment.py and training_script.py.

1. Install the necessary libraries and dependencies for your simulation environment.
2. Build your environment.py module (see Section 3.2.2). This file must contain the logic to create, configure, and wrap your environment. It should return enough information to confirm the environment's interface — at minimum, the observation shape and the action space.
3. In your training_script.py, the initialization section must print the following information to the console when it starts up:

- **The Observation (State) Space** — including shape, data type, and value ranges.
- **The Action Space** — including whether it is discrete or continuous and its dimensions.
- **The device** being used (CPU, CUDA, MPS).
- Confirmation that at least one environment step executed successfully.

Important: You must capture the console output from running your training script and include it in your Progress Report. This confirms the exact nature of the problem you are solving and verifies your Phase 1 claims.

Non-Gymnasium Environments
Not all projects use a standard Gymnasium-style API. If your project uses an external simulator (e.g., Assetto Corsa, SUMO, CARLA, a custom physics engine), your `environment.py` should still expose the equivalent information: what the agent observes, what actions it can take, and how it interacts with the simulator. Include comments explaining the API equivalences.
For ROS 2/Gazebo projects, demonstrate that your ROS node can launch the simulation, receive sensor data (observations), and publish a command (action). Document how you plan to wrap this into a training-compatible interface.

3.2. Task 2: Baseline Learning Agent Skeleton

Based on your Phase 1 proposal, create the code skeleton for your base learning algorithm. These files should contain the classes, methods, and module structure you plan to implement. The methods themselves may contain only pass or minimal placeholder logic.

Your implementation approach will depend on the project you proposed. Some projects call for a from-scratch implementation where you write the RL algorithm yourself (networks, update rules, buffers) using PyTorch or TensorFlow. Other projects may use an existing RL library (e.g., Stable Baselines3, RLlib, CleanRL) and focus the effort on environment design, integration, or analysis. Both approaches are acceptable, but either way your code must be modular, well-documented, and clearly structured — not a single monolithic notebook or script.

3.2.1. Documentation Requirements

A critical part of this phase is professional documentation. Each class and method in your skeleton must include a complete Python docstring that clearly describes:

1. The purpose of the function/method.
2. Its parameters (with types).
3. What it returns (with type).

Example of Required Documentation Style:

```python
def choose_action(self, state):
    """
    Chooses an action for a given state using the epsilon-greedy policy.

    Args:
        state (object): The current state representation from the environment.

    Returns:
        int: The chosen action (e.g., 0, 1, 2, or 3).
    """
    pass
```

3.2.2. Code Structure & Naming Conventions

Your project must be organized into modular, single-responsibility files. The table below shows the recommended structure, based on the reference implementation provided with this assignment. Some files have fixed names that should be the same across all projects, while others are named after your specific algorithm or network type.
Example Skeleton: `dqn_agent.py`

File	Naming Rule	Responsibility
`config.yaml`	Fixed name	All hyperparameters, environment settings, file paths, and execution modes. Avoids hard-coding values in Python files.
`environment.py`	Fixed name	Environment creation, wrappers, and preprocessing (e.g., frame stacking, observation normalization, reward shaping, or a wrapper around an external simulator).
`{algorithm}_agent.py`	Named after algorithm	The RL agent class: action selection, learning/update logic, exploration scheduling, model save/load. Examples: `dqn_agent.py`, `ppo_agent.py`, `sac_agent.py`.
`{type}_network.py`	Named after network	Neural network architecture(s). Examples: `q_network.py`, `actor_critic.py`, `policy_network.py`.
`{type}_buffer.py`	Named after type	Experience storage, if applicable. Examples: `replay_buffer.py` for off-policy, `rollout_buffer.py` for on-policy methods.
`training_script.py`	Fixed name	Main entry point: training loop, evaluation, deployment, and orchestration logic.
`utils.py`	Fixed name	Shared utilities: config loading, plotting, logging, seed management, and helper functions.
`README.md`	Fixed name	Project overview, installation/setup instructions, how to run, and configuration reference.
`{algorithm}_results/`	Named after algorithm	Output directory for saved model weights, training history, and plots. Examples: `dqn_results/`, `ppo_results/`.

Why this naming matters: In Phase 3, you will add your advanced algorithm alongside the base. Algorithm-specific naming makes it immediately clear which file belongs to which implementation. For example, a project comparing DQN and Rainbow DQN would contain both `dqn_agent.py` and `rainbow_agent.py`, with results in `dqn_results/` and `rainbow_results/` respectively. They may share the same `q_network.py`, `environment.py`, and `config.yaml`.

Reference structure (Naive DQN — provided in Chapter 4):

```
Naive DQN/
├── config.yaml            # All hyperparameters and settings
├── environment.py         # Atari preprocessing and wrappers
├── q_network.py           # CNN architecture (DeepMind 2015)
├── dqn_agent.py           # Naive DQN agent
├── training_script.py     # Training loop and main entry point
├── utils.py               # Config loading, plotting, deployment
├── README.md
└── naive_dqn_results/     # Model weights, history, plots
```

Example: PPO for Bipedal Walker

```
PPO BipedalWalker/
├── config.yaml
├── environment.py         # BipedalWalker setup and wrappers
├── actor_critic.py        # Actor-Critic network architecture
├── ppo_agent.py           # PPO agent with clipped objective
├── rollout_buffer.py      # On-policy trajectory storage
├── training_script.py
├── utils.py
├── README.md
└── ppo_results/
```

Example: DQN for Algorithmic Trading (custom environment)

```
Trading Agent/
├── config.yaml
├── environment.py         # Custom Gym env for stock trading
├── q_network.py           # Q-network (MLP for state vectors)
├── dqn_agent.py           # DQN agent
├── replay_buffer.py       # Experience replay
├── training_script.py
├── utils.py
├── README.md
└── dqn_results/
```

Example: Autonomous Racing with Assetto Corsa

```
Autonomous Racing/
├── config.yaml
├── environment.py         # AssettoCorsaGym wrapper
├── actor_critic.py        # Actor-Critic network
├── ppo_agent.py           # PPO agent for continuous control
├── rollout_buffer.py
├── training_script.py
├── utils.py
├── README.md
└── ppo_results/
```

Example: ROS 2 / Gazebo Navigation

```
ROS2 Navigation/
├── config.yaml
├── environment.py         # Gym wrapper around ROS topics
├── sensor_processing.py   # LiDAR / camera preprocessing
├── policy_network.py      # Policy network architecture
├── ppo_agent.py           # PPO agent
├── rollout_buffer.py
├── training_script.py
├── utils.py
├── launch/                # ROS 2 launch files
├── README.md
└── ppo_results/
```

Using RL Libraries

Some projects may use an existing RL library rather than implement the algorithm from scratch. This is acceptable, especially when the primary challenge is environment design or system integration. However, your code must still follow the structure above. Your `{algorithm}_agent.py` file should contain a configuration class or wrapper that documents all library parameters you are using and why. Your `training_script.py` must still show how you instantiate, train, save, and evaluate the model.

You are still expected to understand the algorithm you are using and be able to explain it during your Phase 3 presentation.

3.2.3. Required Skeleton Components

Regardless of your chosen algorithm, environment, or implementation approach, your skeleton must include documented stubs (or equivalents) for the following core responsibilities. The exact method names can vary to fit your algorithm:

Criteria	Description
Initialization	Constructor that accepts hyperparameters, sets up networks, optimizer, loss function, and any buffers or storage structures.
Action Selection	Method to choose an action given a state. Must support both exploration (during training) and exploitation (during deployment) modes.
Learning / Update	Method to update the agent's parameters from experience. This varies by algorithm: single-step TD update, batch replay, policy gradient step, etc.
Experience Storage	If applicable: method or class to store transitions in a replay buffer, rollout buffer, or similar structure.
Exploration Schedule	Method to manage exploration over time (e.g., epsilon decay for value-based methods, entropy coefficient for policy gradient, noise schedule for continuous control).
Save / Load Model	Methods to persist and restore model weights for checkpointing and deployment.
Training Loop	A main training function (in training_script.py) that runs episodes, calls the agent, logs metrics, and saves results.
Deployment / Evaluation	A function that loads a trained model and runs it in pure exploitation mode for evaluation.

You may need additional classes or modules depending on your project (e.g., a custom Gymnasium wrapper for an external simulator, a reward shaping module, sensor preprocessing pipelines, ROS service/topic handlers, etc.). These must also be documented.
This skeleton will serve as your "**to-do list**" for Phase 3 and proves your code structure is sound.

4. AI Usage Policy

You are permitted and encouraged to use AI tools (e.g., ChatGPT, Claude, GitHub Copilot, Gemini, etc.) to assist you throughout this project. However, transparency and accountability are required.
You must include an AI Usage Log as an appendix to every phase submission. This log must contain:

- **Tool Identification:** The name and version of each AI tool used (e.g., "ChatGPT-4o", "Claude Sonnet 4", "GitHub Copilot").
- **Purpose:** A brief description of how the tool was used (e.g., "Used to debug a tensor shape mismatch", "Used to generate boilerplate for the replay buffer class", "Used to help write the Gymnasium wrapper for our simulator").
- **Full Prompt Log:** A complete list of all prompts (queries) you submitted to the AI tool, in order. You may paraphrase very long conversations, but the substance of every prompt must be captured.

- **What You Changed:** A note on whether you used the AI output as-is or modified it, and what changes you made.

Academic Integrity
Using AI without documenting it in your AI Usage Log will be treated as a violation of academic integrity. You are fully responsible for understanding and being able to explain every line of code you submit.
During Phase 3 (the final demo/presentation), you may be asked to explain any part of your implementation, including code that was generated or assisted by AI tools.

5. Submission Details

You must submit **two (2)** items to the "Project Phase 2" portal:

1. **PDF Progress Report (1–2 pages):** A brief report containing:
   - A confirmation that your environment is installed and running.
   - **The console output** from running your training script, showing the observation space, action space, and device information (as described in Task 1).
   - A brief discussion of whether these confirmed spaces match your Phase 1 proposal, and any challenges you foresee.
   - **Your implementation approach** — briefly state whether you are implementing the algorithm from scratch or using a library, and justify the choice based on your project.
   - **The AI Usage Log** (as described in Section 4), included as an appendix.
   - **Team Contribution Efforts** — each member must outline their specific contributions to this phase in short bullet points.
2. **Code Archive (.zip file):** A single .zip file containing:
   - All your skeleton code files from Task 2, following the naming conventions in Section 3.2.2.
   - config.yaml with all hyperparameters and settings.
   - README.md with setup instructions and a brief project description.
   - A requirements.txt or equivalent file listing all Python dependencies.

6. Grading Criteria

This phase is graded on completion and correctness, not on agent performance.

Criteria	Description	Points
Environment & API Confirmation	Report includes the console output from the training script showing observation space, action space, and device. Spaces are clearly identified and discussed relative to the Phase 1 proposal.	25
Agent Skeleton Quality	Code skeleton is well-structured, modular, and appropriate for the algorithm proposed in Phase 1. All classes and methods have complete docstrings.	40
Code Organization	Clear file structure following the naming conventions in Section 3.2.2. Includes config.yaml, README.md, requirements.txt, and logical module separation.	15
AI Usage Log	Complete and honest documentation of AI tool usage, including all prompts and modifications.	10
Professionalism	Report is clear and well-written. All files are correctly packaged. Team contribution efforts are included.	10
Total		100

7. Team Collaboration

This is a group project. All team members are expected to contribute to this phase. Your progress report must include a Team Contribution Efforts section in which each member outlines their specific contributions in short bullet points. Failure to include this section will result in a deduction under the Professionalism criterion.